PARTE 1 - Infraestrutura

In [15]:
```
# importando as bibliotecas necessárias

import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot  as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn_extra.cluster import KMedoids
from scipy.cluster.hierarchy import dendrogram
```

In [16]:
```
#informando a versão do python e também o ambiente virtual utilizado
print(f'A versão do python instalada é: {sys.version}\nRodando no ambiente virtual:
```

```
A versão do python instalada é: 3.13.0 | packaged by Anaconda, Inc. | (main, Oct  7
2024, 21:21:52) [MSC v.1929 64 bit (AMD64)]
Rodando no ambiente virtual: c:\Users\Alan\miniconda3\envs\clusterizacao
```

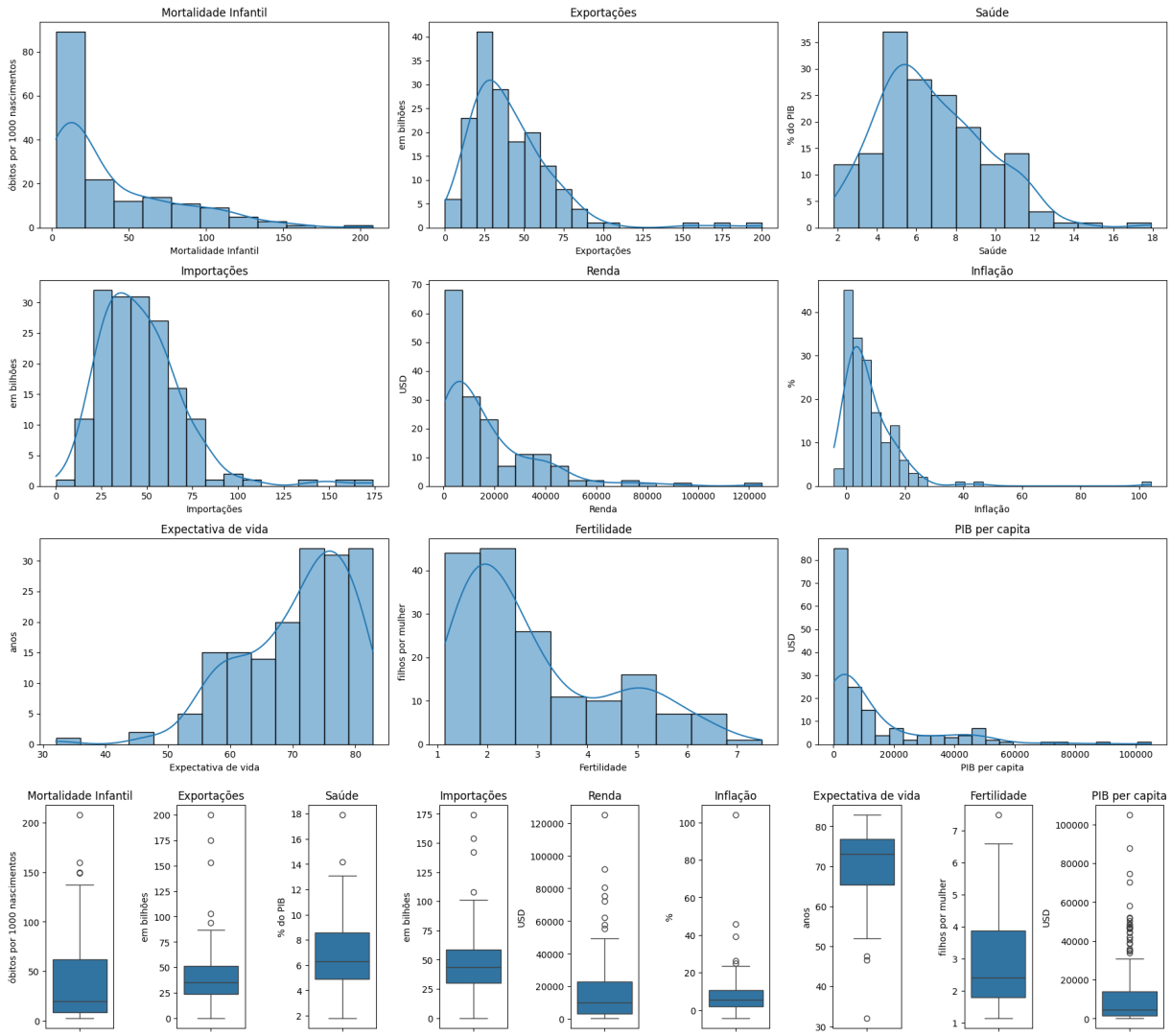
Este arquivo, em conjunto com o dataset, requirements.txt e um screenshot do ambiente de desenvolvimento estão no github: https://github.com/AlansAlonso/PG-IA

PARTE 2 - Escolha de base de dados

In [17]:
```
# lendo os dados obtidos jpá baixados da base de dados indicada
dataset = pd.read_csv('country-data.csv', sep=',', decimal='.')

# renomeando as colunas para melhor usabilidade e especificando as unidades
novos_nomes = ["País", "Mortalidade Infantil", "Exportações", "Saúde","Importações"
unidades = ['óbitos por 1000 nascimentos', 'em bilhões', '% do PIB', 'em bilhões',

# atualizar os nomes das colunas diretamente
dataset.columns = novos_nomes

# quantos países existem no dataset?
print("Quantidade de Paises na base de dados: ", len(dataset))

# checando se há dados faltantes
if dataset.isna().sum().sum() > 0:
    print(f'Existem {dataset.isna().sum()} dados faltantes no dataset')
else:
    print("Não há dados faltantes no dataset.")

# checando se há dados duplicados
if dataset.duplicated().sum().sum() > 0:
    print(f'Existem {dataset.duplicated().sum()} dados duplicados no dataset')
else:
```

```
    print("Não há dados duplicados no dataset.")


# verificando os tipos de cada dado
print("Tipo de dados:")
print(dataset.dtypes)

# selecionando apenas as colunas numéricas para plotagem
colunas_numericas = dataset.select_dtypes(include=['float64', 'int64']).columns
plt.figure(figsize=(18, 12))

# plotando um histograma
for index, value in enumerate(colunas_numericas):
    plt.subplot(3, 3, index + 1)
    sns.histplot(dataset[value], kde=True)
    plt.title(value)
    plt.ylabel(unidades[index])
plt.tight_layout()

# plotando um boxplot
plt.figure(figsize=(18,4))
for index, column in enumerate(colunas_numericas):
    plt.subplot(1, 9, index+1)
    sns.boxplot(dataset[column])
    plt.title(column)
    plt.ylabel(unidades[index])
plt.tight_layout()
```

```
Quantidade de Paises na base de dados:  167
Não há dados faltantes no dataset.
Não há dados duplicados no dataset.
Tipo de dados:
País                     object
Mortalidade Infantil    float64
Exportações             float64
Saúde                   float64
Importações             float64
Renda                     int64
Inflação                float64
Expectativa de vida     float64
Fertilidade             float64
PIB per capita            int64
dtype: object
```



In [18]:
```
# detectando e identificando os outliers nos boxplots

def detectar_outliers(df, coluna):
    # determinando os quartis
    Q1 = df[coluna].quantile(0.25)
    Q3 = df[coluna].quantile(0.75)
    IQR = Q3 - Q1

    # limites superior e inferior para outliers
    limite_inferior = Q1 - 1.5 * IQR
    limite_superior = Q3 + 1.5 * IQR

    # identificando os outliers
    outliers = df[(df[coluna] < limite_inferior) | (df[coluna] > limite_superior)]

    return outliers

# buscando os outliers em cada uma das colunas do dataset
for column in colunas_numericas:
    outliers = detectar_outliers(dataset, column)

    if not outliers.empty:
        print(f'Outliers para a variável {column}:')
```

```
        for _, row in outliers.iterrows():
            print(f"País: {row['País']} - Valor da variável: {row[column]}")
        print('-' * 60)
```

```
Outliers para a variável Mortalidade Infantil:
País: Central African Republic - Valor da variável: 149.0
País: Chad - Valor da variável: 150.0
País: Haiti - Valor da variável: 208.0
País: Sierra Leone - Valor da variável: 160.0
------------------------------------------------------------
Outliers para a variável Exportações:
País: Ireland - Valor da variável: 103.0
País: Luxembourg - Valor da variável: 175.0
País: Malta - Valor da variável: 153.0
País: Seychelles - Valor da variável: 93.8
País: Singapore - Valor da variável: 200.0
------------------------------------------------------------
Outliers para a variável Saúde:
País: Micronesia, Fed. Sts. - Valor da variável: 14.2
País: United States - Valor da variável: 17.9
------------------------------------------------------------
Outliers para a variável Importações:
País: Luxembourg - Valor da variável: 142.0
País: Malta - Valor da variável: 154.0
País: Seychelles - Valor da variável: 108.0
País: Singapore - Valor da variável: 174.0
------------------------------------------------------------
Outliers para a variável Renda:
País: Brunei - Valor da variável: 80600
País: Kuwait - Valor da variável: 75200
País: Luxembourg - Valor da variável: 91700
País: Norway - Valor da variável: 62300
País: Qatar - Valor da variável: 125000
País: Singapore - Valor da variável: 72100
País: Switzerland - Valor da variável: 55500
País: United Arab Emirates - Valor da variável: 57600
------------------------------------------------------------
Outliers para a variável Inflação:
País: Equatorial Guinea - Valor da variável: 24.9
País: Mongolia - Valor da variável: 39.2
País: Nigeria - Valor da variável: 104.0
País: Timor-Leste - Valor da variável: 26.5
País: Venezuela - Valor da variável: 45.9
------------------------------------------------------------
Outliers para a variável Expectativa de vida:
País: Central African Republic - Valor da variável: 47.5
País: Haiti - Valor da variável: 32.1
País: Lesotho - Valor da variável: 46.5
------------------------------------------------------------
Outliers para a variável Fertilidade:
País: Niger - Valor da variável: 7.49
------------------------------------------------------------
Outliers para a variável PIB per capita:
País: Australia - Valor da variável: 51900
País: Austria - Valor da variável: 46900
País: Belgium - Valor da variável: 44400
País: Brunei - Valor da variável: 35300
País: Canada - Valor da variável: 47400
País: Denmark - Valor da variável: 58000
País: Finland - Valor da variável: 46200
```

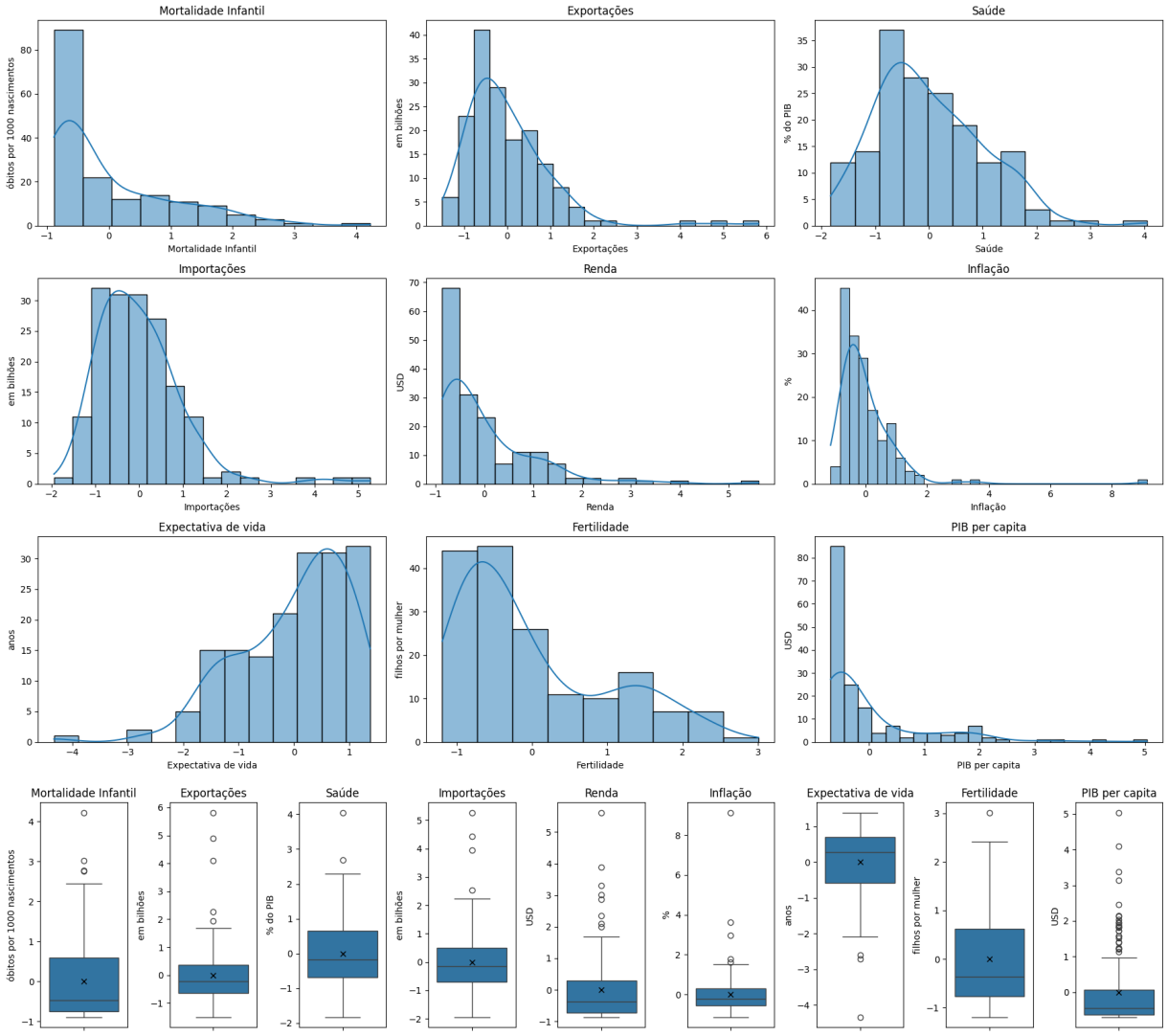
```
País: France - Valor da variável: 40600
País: Germany - Valor da variável: 41800
País: Iceland - Valor da variável: 41900
País: Ireland - Valor da variável: 48700
País: Italy - Valor da variável: 35800
País: Japan - Valor da variável: 44500
País: Kuwait - Valor da variável: 38500
País: Luxembourg - Valor da variável: 105000
País: Netherlands - Valor da variável: 50300
País: New Zealand - Valor da variável: 33700
País: Norway - Valor da variável: 87800
País: Qatar - Valor da variável: 70300
País: Singapore - Valor da variável: 46600
País: Sweden - Valor da variável: 52100
País: Switzerland - Valor da variável: 74600
País: United Arab Emirates - Valor da variável: 35000
País: United Kingdom - Valor da variável: 38900
País: United States - Valor da variável: 48400
------------------------------------------------------------
```

Com os dados obtidos no boxplot anterior, achei que seria interessante buscar que países estavam sendo outliers em quais variáveis. Em especial, alguns países com características muito específicas como Luxembourg foram outliers recorrentes, sugerindo um padrão de exceção. Além disso, vê-se que em especial o PIB per capita teve muitos outliers, também podemos analisar a importância dessa coluna em nossa base de dados.

Agora, para que possamos utilizar esses dados em uma clusterização, precisamos impedir que a diferença de escala atrapalhe o nosso projeto. Por exemplo, enquanto a inflação tem uma ou duas ordens de grandeza, valores como o PIB per capita podem chegar a 5 ordens de grandeza. Para resolver isso, podemos fazer uma normalização, com o MinMaxScaler, que ajusta todos os números entre 0 e 1, ou uma padronização com StandardScaler, onde os números serão ajustados de forma que a média será 0 e o desvio padrão igual a 1

In [19]:
```
# Foi decidido por uma padronização dos dados utilizando StandardScaler em detrimen
# dentre os motivos a falta de necessidade de normalizar os valores entre 0 e 1 e a
# superior em muitos desses dados, que dessem um sentido físico à possível normaliz
# Além disso, StandardScaler é recomendada no uso de k-means

scaler = StandardScaler()
scaler.fit(dataset[colunas_numericas])
dataset_tratado = scaler.transform(dataset[colunas_numericas])

# nomeando as colunas do dataset tratado
dataset_tratado  = pd.DataFrame(dataset_tratado, columns=colunas_numericas)

# fazendo o plot dos novos valores padronizados
plt.figure(figsize=(18, 12))

for index, value in enumerate(dataset_tratado[colunas_numericas]):
    plt.subplot(3, 3, index + 1)
    sns.histplot(dataset_tratado[value], kde=True)
    plt.title(value)
    plt.ylabel(unidades[index])
```

```
plt.tight_layout()

plt.figure(figsize=(18,4))
for index, column in enumerate(dataset_tratado[colunas_numericas]):
    plt.subplot(1, 9, index+1)
    sns.boxplot(dataset_tratado[column],showmeans=True,meanprops={"marker":"x", "ma
    plt.title(column)
    plt.ylabel(unidades[index])
plt.tight_layout()
```



Especialmente interessante observar nestes gráficos de boxplot que a média, marcada por um "x" é sempre igual a 0, por definição do uso de StandardScaler.

PARTE 3 - Clusterização

In [20]:
```
# realizar a clusterização dos países com base em 3 grupos e definindo uma seed

kmeans = KMeans(n_clusters=3,random_state=11)
modelo = kmeans.fit(dataset_tratado)
grupos_kmeans = kmeans.predict(dataset_tratado)
```

```python
# determinação dos clusters
clusters_paises = pd.DataFrame(grupos_kmeans, columns={0: 'Agrupamento'})
clusters_paises = clusters_paises.rename(columns={0: 'Agrupamento'})

# adicionando novamente o nome dos países ao resultado do k-means
clusters_paises['Pais'] = dataset['País']

clusters_paises = clusters_paises.sort_values(by='Agrupamento')
pd.set_option('display.max_rows', None)
print(clusters_paises)
```

	Agrupamento	Pais
1	0	Albania
2	0	Algeria
5	0	Argentina
4	0	Antigua and Barbuda
6	0	Armenia
13	0	Barbados
10	0	Bahamas
9	0	Azerbaijan
14	0	Belarus
12	0	Bangladesh
21	0	Botswana
22	0	Brazil
20	0	Bosnia and Herzegovina
19	0	Bolivia
18	0	Bhutan
16	0	Belize
30	0	Cape Verde
24	0	Bulgaria
27	0	Cambodia
51	0	Estonia
61	0	Grenada
62	0	Guatemala
52	0	Fiji
57	0	Georgia
41	0	Croatia
47	0	Egypt
46	0	Ecuador
45	0	Dominican Republic
39	0	Costa Rica
35	0	Colombia
34	0	China
33	0	Chile
48	0	El Salvador
102	0	Moldova
101	0	Micronesia, Fed. Sts.
100	0	Mauritius
103	0	Mongolia
107	0	Myanmar
105	0	Morocco
104	0	Montenegro
109	0	Nepal
121	0	Poland
125	0	Russia
124	0	Romania
117	0	Panama
118	0	Paraguay
119	0	Peru
115	0	Oman
127	0	Samoa
83	0	Kyrgyz Republic
86	0	Lebanon
85	0	Latvia
92	0	Macedonia, FYR
95	0	Malaysia
90	0	Lithuania

89	0	Libya
78	0	Jordan
79	0	Kazakhstan
76	0	Jamaica
71	0	Iran
69	0	India
70	0	Indonesia
67	0	Hungary
65	0	Guyana
120	0	Philippines
96	0	Maldives
151	0	Tonga
148	0	Thailand
156	0	Ukraine
154	0	Turkmenistan
153	0	Turkey
152	0	Tunisia
146	0	Tajikistan
143	0	Suriname
141	0	St. Vincent and the Grenadines
140	0	Sri Lanka
136	0	Solomon Islands
164	0	Vietnam
162	0	Vanuatu
163	0	Venezuela
160	0	Uruguay
161	0	Uzbekistan
130	0	Serbia
128	0	Saudi Arabia
131	0	Seychelles
99	1	Mauritania
113	1	Nigeria
64	1	Guinea-Bissau
80	1	Kenya
66	1	Haiti
72	1	Iraq
94	1	Malawi
84	1	Lao
97	1	Mali
106	1	Mozambique
116	1	Pakistan
126	1	Rwanda
112	1	Niger
108	1	Namibia
93	1	Madagascar
87	1	Lesotho
81	1	Kiribati
166	1	Zambia
142	1	Sudan
129	1	Senegal
137	1	South Africa
149	1	Timor-Leste
155	1	Uganda
150	1	Togo
50	1	Eritrea
49	1	Equatorial Guinea

88	1	Liberia
37	1	Congo, Dem. Rep.
38	1	Congo, Rep.
36	1	Comoros
17	1	Benin
3	1	Angola
0	1	Afghanistan
26	1	Burundi
25	1	Burkina Faso
31	1	Central African Republic
28	1	Cameroon
59	1	Ghana
32	1	Chad
40	1	Cote d'Ivoire
55	1	Gabon
63	1	Guinea
56	1	Gambia
132	1	Sierra Leone
147	1	Tanzania
165	1	Yemen
123	2	Qatar
122	2	Portugal
98	2	Malta
82	2	Kuwait
91	2	Luxembourg
68	2	Iceland
77	2	Japan
114	2	Norway
111	2	New Zealand
110	2	Netherlands
53	2	Finland
58	2	Germany
60	2	Greece
29	2	Canada
44	2	Denmark
42	2	Cyprus
23	2	Brunei
7	2	Australia
8	2	Austria
11	2	Bahrain
15	2	Belgium
54	2	France
43	2	Czech Republic
74	2	Israel
75	2	Italy
73	2	Ireland
138	2	South Korea
133	2	Singapore
135	2	Slovenia
159	2	United States
158	2	United Kingdom
157	2	United Arab Emirates
145	2	Switzerland
144	2	Sweden
134	2	Slovak Republic
139	2	Spain

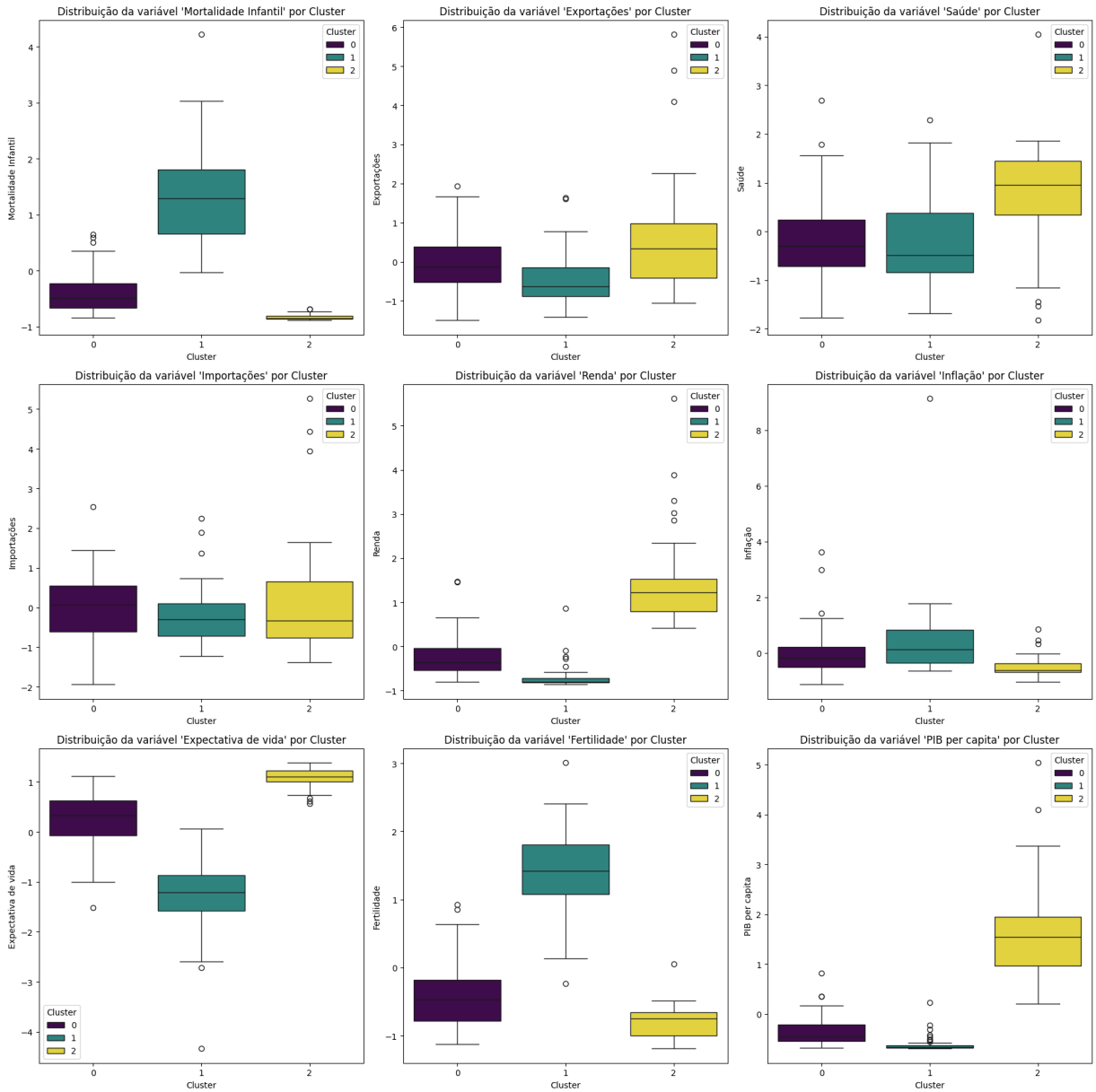
In [21]:
```
### com base nos clusters observados podemos encontrar certos padrões e nomear cada
# grupo 0 - Economias Emergentes
# grupo 1 - Economias em Vulnerabilidade
# grupo 2 - Economias Avançadas

nomes_grupos = {
    0: "Economias Emergentes",
    1: "Economias em Vulnerabilidade",
    2: "Economias Avançadas"
}

clusters_paises['Grupo'] = clusters_paises['Agrupamento'].map(nomes_grupos)


### buscando o país que melhor representa o seu cluster, com base naquele que tem a
# essa distância será dada pela distância euclidiana, adequada para casos onde os d

# calculando as distâncias de cada (país) para os centroides dos clusters
distancia_centros = kmeans.transform(dataset_tratado)
distancias = pd.DataFrame(distancia_centros)
pais_mais_representativo_lista = []

# iterando em cada centroide para achar o país mais próximo
for alternativa in range(kmeans.n_clusters):

    # obtendo as distâncias de todos os países para o centroide do cluster atual da
    centroide_distancia = distancias.iloc[:, alternativa].tolist()
    pais_proximo_index = np.argmin(centroide_distancia)

    # adicionando ele como mais representativo para este cluster específico na list
    pais_mais_representativo_lista.append({
        "Grupo": nomes_grupos[alternativa],
        "Pais que mais representa": clusters_paises.loc[pais_proximo_index, 'Pais']
        "Distancia": centroide_distancia[pais_proximo_index]
    })

# converter a lista para DataFrame
pais_mais_representativo = pd.DataFrame(pais_mais_representativo_lista)

# exibindo o país mais representativo de cada cluster
print("Pontos mais próximos dos centroides:\n", pais_mais_representativo)
```

```
Pontos mais próximos dos centroides:
                          Grupo Pais que mais representa  Distancia
0          Economias Emergentes                 Suriname   0.731590
1  Economias em Vulnerabilidade                   Guinea   0.797841
2           Economias Avançadas                  Iceland   0.731764
```

Com isso em mãos, fiz um novo boxplot para entender como cada variável se distribui dentro de cada cluster

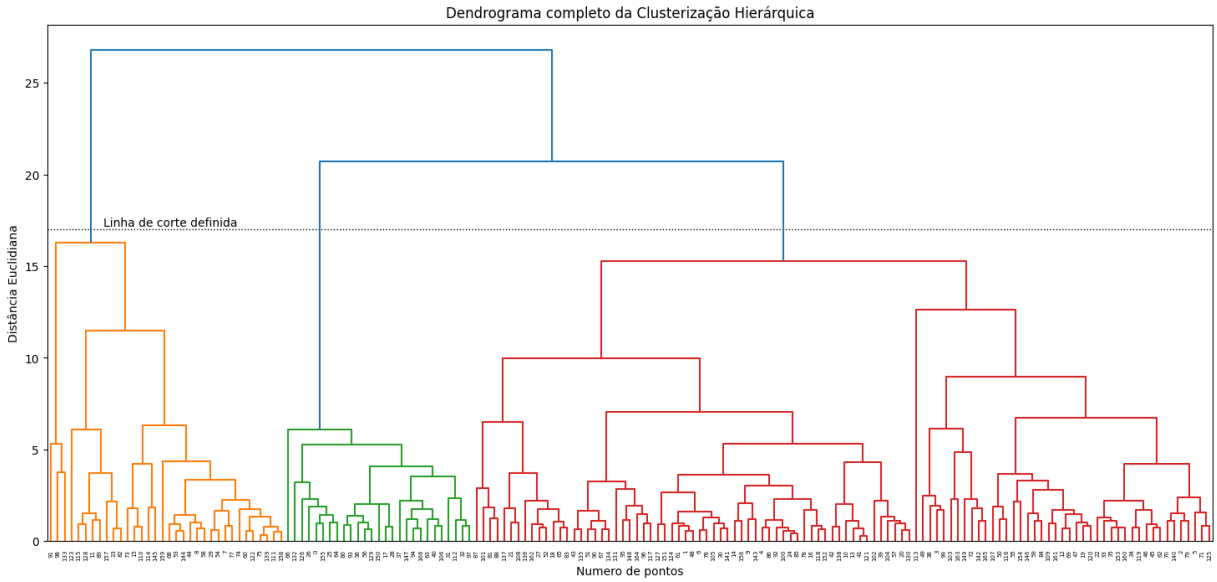
In [22]:
```
fig, axes = plt.subplots(3, 3, figsize=(18, 18))

# criando boxplots para todas as variáveis numéricas
for i in range(3):
```

```
    for j in range(3):
        ax = axes[i, j]
        #selecionando a coluna específica
        coluna = dataset_tratado.columns[i * 3 + j]
        sns.boxplot(
            data=dataset_tratado.assign(Cluster=clusters_paises['Agrupamento']),
            x='Cluster',
            y=coluna,
            palette="viridis",
            ax=ax,
            hue='Cluster',
        )
        ax.set_title(f"Distribuição da variável '{coluna}' por Cluster")
        ax.set_xlabel("Cluster")
        ax.set_ylabel(coluna)

# ajustando espaçamento para não sobrepor títulos e labels
plt.tight_layout()
plt.show()
```



Com esses gráficos podemos visualizar melhor a categorização de cada cluster entre economia emergente, em vulnerabilidade e avançada. Conseguimos também observar as variáveis que tem maior influência nessa clusterização (alto grau de separação entre os clusters), como expectativa de vida, renda e mortalidade infantil.

In [23]:
```
# definindo o plot do dendograma da clusterização hierarquica
def plot_dendrogram(model, **kwargs):

    # inicializando a array para contar os filhos em cada nó
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)

    # Iterar sobre os nós da árvore e calcular os filhos de forma eficiente
    for i, (left_child, right_child) in enumerate(model.children_):
        left_count = 1 if left_child < n_samples else counts[left_child - n_samples
        right_count = 1 if right_child < n_samples else counts[right_child - n_samp
        counts[i] = left_count + right_count

    # matriz de linkage
    linkage_matrix = np.column_stack([model.children_, model.distances_, counts]).a

    # retorno
    return dendrogram(linkage_matrix, **kwargs)

# realizando a clusterização hierarquica
clusterizacao_hierarquica = AgglomerativeClustering(n_clusters=3, metric='euclidean
model = clusterizacao_hierarquica.fit(dataset_tratado)

# plotando o dendograma
plt.figure(figsize=(18, 8))
plt.title('Dendrograma completo da Clusterização Hierárquica')
plt.ylabel('Distância Euclidiana')
plt.xlabel("Numero de pontos")
plt.hlines(y=17,xmin=0,xmax=2000,lw=1,linestyles='dotted',colors='black' )
plt.text(x=80,y=17.2,s='Linha de corte definida',fontsize=10)
dendro=plot_dendrogram(model)
plt.show()
```



Algo muito interessante que pode ser aferido deste gráfico, que já era uma impressão no Kmeans se torna bem mais evidente com o dendograma, é que a distância entre os países emergentes e os países com economia vulnerável é menor do que a entre os emergentes e os com economia avançada.

In [24]:
```
cluster_resultado = clusterizacao_hierarquica.fit_predict(dataset_tratado)

# criando o DataFrame com o resultado da clusterização
resultado_hierarquico = pd.DataFrame(cluster_resultado)
resultado_hierarquico = resultado_hierarquico.rename(columns={0: 'Agrupamento'})
resultado_hierarquico['Pais'] = dataset['País']

# ordenando o DataFrame pelos valores do grupo
resultado_hierarquico = resultado_hierarquico.sort_values(by='Agrupamento')

# exibindo o DataFrame com o resultado da clusterização hierárquica
pd.set_option('display.max_rows', None)
print(resultado_hierarquico)
```

	Agrupamento	Pais
7	0	Australia
15	0	Belgium
11	0	Bahrain
8	0	Austria
29	0	Canada
23	0	Brunei
60	0	Greece
44	0	Denmark
54	0	France
53	0	Finland
58	0	Germany
77	0	Japan
73	0	Ireland
74	0	Israel
75	0	Italy
68	0	Iceland
82	0	Kuwait
98	0	Malta
111	0	New Zealand
110	0	Netherlands
123	0	Qatar
122	0	Portugal
114	0	Norway
115	0	Oman
91	0	Luxembourg
89	0	Libya
159	0	United States
158	0	United Kingdom
157	0	United Arab Emirates
139	0	Spain
133	0	Singapore
128	0	Saudi Arabia
144	0	Sweden
145	0	Switzerland
59	1	Ghana
57	1	Georgia
51	1	Estonia
50	1	Eritrea
35	1	Colombia
34	1	China
55	1	Gabon
52	1	Fiji
47	1	Egypt
42	1	Cyprus
38	1	Congo, Rep.
43	1	Czech Republic
41	1	Croatia
39	1	Costa Rica
62	1	Guatemala
14	1	Belarus
19	1	Bolivia
16	1	Belize
21	1	Botswana
20	1	Bosnia and Herzegovina
22	1	Brazil

18	1	Bhutan
46	1	Ecuador
45	1	Dominican Republic
30	1	Cape Verde
4	1	Antigua and Barbuda
10	1	Bahamas
6	1	Armenia
5	1	Argentina
1	1	Albania
124	1	Romania
119	1	Peru
107	1	Myanmar
108	1	Namibia
118	1	Paraguay
88	1	Liberia
90	1	Lithuania
96	1	Maldives
109	1	Nepal
103	1	Mongolia
104	1	Montenegro
105	1	Morocco
49	1	Equatorial Guinea
48	1	El Salvador
61	1	Grenada
72	1	Iraq
92	1	Macedonia, FYR
87	1	Lesotho
79	1	Kazakhstan
81	1	Kiribati
86	1	Lebanon
85	1	Latvia
83	1	Kyrgyz Republic
84	1	Lao
95	1	Malaysia
76	1	Jamaica
71	1	Iran
70	1	Indonesia
78	1	Jordan
65	1	Guyana
69	1	India
67	1	Hungary
127	1	Samoa
125	1	Russia
121	1	Poland
120	1	Philippines
113	1	Nigeria
117	1	Panama
116	1	Pakistan
100	1	Mauritius
102	1	Moldova
101	1	Micronesia, Fed. Sts.
99	1	Mauritania
138	1	South Korea
142	1	Sudan
143	1	Suriname
154	1	Turkmenistan

153	1	Turkey
130	1	Serbia
136	1	Solomon Islands
134	1	Slovak Republic
135	1	Slovenia
131	1	Seychelles
137	1	South Africa
141	1	St. Vincent and the Grenadines
140	1	Sri Lanka
27	1	Cambodia
24	1	Bulgaria
33	1	Chile
9	1	Azerbaijan
13	1	Barbados
12	1	Bangladesh
3	1	Angola
2	1	Algeria
165	1	Yemen
152	1	Tunisia
149	1	Timor-Leste
164	1	Vietnam
163	1	Venezuela
151	1	Tonga
146	1	Tajikistan
148	1	Thailand
156	1	Ukraine
162	1	Vanuatu
161	1	Uzbekistan
160	1	Uruguay
66	2	Haiti
112	2	Niger
97	2	Mali
106	2	Mozambique
31	2	Central African Republic
0	2	Afghanistan
25	2	Burkina Faso
26	2	Burundi
17	2	Benin
80	2	Kenya
93	2	Madagascar
94	2	Malawi
64	2	Guinea-Bissau
28	2	Cameroon
37	2	Congo, Dem. Rep.
36	2	Comoros
40	2	Cote d'Ivoire
32	2	Chad
56	2	Gambia
63	2	Guinea
126	2	Rwanda
155	2	Uganda
147	2	Tanzania
129	2	Senegal
132	2	Sierra Leone
150	2	Togo
166	2	Zambia

In [25]:
```
# diferente do KMeans, na clusterização hierárquica os países com economia avançada
# e Economias em Vulnerabilidade foram o número 2

# convertendo os valores numéricos para o mesmo significado semântico
resultado_hierarquico_comparativo = resultado_hierarquico.replace({0: 2, 1: 0, 2: 1

# criando uma df unificada e fazendo a comparação entre as colunas
comparacao_clusterizacao = resultado_hierarquico_comparativo.sort_values(by='Pais')
comparacao_clusterizacao['K-means'] = clusters_paises.sort_values(by='Pais')['Agrup
comparacao_clusterizacao['Hierárquica'] = resultado_hierarquico_comparativo.sort_va
resultado_comparativo = comparacao_clusterizacao[['Pais', 'K-means', 'Hierárquica']

pd.set_option('display.max_rows', None)
print(resultado_comparativo)

# checando o % de similaridade entre os métodos
similaridades = comparacao_clusterizacao['K-means'] == comparacao_clusterizacao["Hi
total_similaridades = similaridades.sum()

print('-' * 60)
print(f'Os métodos de clusterização tiveram o mesmo resultado em {100*total_similar
```

	Pais	K-means	Hierárquica
0	Afghanistan	1	1
1	Albania	0	0
2	Algeria	0	0
3	Angola	1	0
4	Antigua and Barbuda	0	0
5	Argentina	0	0
6	Armenia	0	0
7	Australia	2	2
8	Austria	2	2
9	Azerbaijan	0	0
10	Bahamas	0	0
11	Bahrain	2	2
12	Bangladesh	0	0
13	Barbados	0	0
14	Belarus	0	0
15	Belgium	2	2
16	Belize	0	0
17	Benin	1	1
18	Bhutan	0	0
19	Bolivia	0	0
20	Bosnia and Herzegovina	0	0
21	Botswana	0	0
22	Brazil	0	0
23	Brunei	2	2
24	Bulgaria	0	0
25	Burkina Faso	1	1
26	Burundi	1	1
27	Cambodia	0	0
28	Cameroon	1	1
29	Canada	2	2
30	Cape Verde	0	0
31	Central African Republic	1	1
32	Chad	1	1
33	Chile	0	0
34	China	0	0
35	Colombia	0	0
36	Comoros	1	1
37	Congo, Dem. Rep.	1	1
38	Congo, Rep.	1	0
39	Costa Rica	0	0
40	Cote d'Ivoire	1	1
41	Croatia	0	0
42	Cyprus	2	0
43	Czech Republic	2	0
44	Denmark	2	2
45	Dominican Republic	0	0
46	Ecuador	0	0
47	Egypt	0	0
48	El Salvador	0	0
49	Equatorial Guinea	1	0
50	Eritrea	1	0
51	Estonia	0	0
52	Fiji	0	0
53	Finland	2	2
54	France	2	2

55	Gabon	1	0
56	Gambia	1	1
57	Georgia	0	0
58	Germany	2	2
59	Ghana	1	0
60	Greece	2	2
61	Grenada	0	0
62	Guatemala	0	0
63	Guinea	1	1
64	Guinea-Bissau	1	1
65	Guyana	0	0
66	Haiti	1	1
67	Hungary	0	0
68	Iceland	2	2
69	India	0	0
70	Indonesia	0	0
71	Iran	0	0
72	Iraq	1	0
73	Ireland	2	2
74	Israel	2	2
75	Italy	2	2
76	Jamaica	0	0
77	Japan	2	2
78	Jordan	0	0
79	Kazakhstan	0	0
80	Kenya	1	1
81	Kiribati	1	0
82	Kuwait	2	2
83	Kyrgyz Republic	0	0
84	Lao	1	0
85	Latvia	0	0
86	Lebanon	0	0
87	Lesotho	1	0
88	Liberia	1	0
89	Libya	0	2
90	Lithuania	0	0
91	Luxembourg	2	2
92	Macedonia, FYR	0	0
93	Madagascar	1	1
94	Malawi	1	1
95	Malaysia	0	0
96	Maldives	0	0
97	Mali	1	1
98	Malta	2	2
99	Mauritania	1	0
100	Mauritius	0	0
101	Micronesia, Fed. Sts.	0	0
102	Moldova	0	0
103	Mongolia	0	0
104	Montenegro	0	0
105	Morocco	0	0
106	Mozambique	1	1
107	Myanmar	0	0
108	Namibia	1	0
109	Nepal	0	0
110	Netherlands	2	2

111	New Zealand	2	2
112	Niger	1	1
113	Nigeria	1	0
114	Norway	2	2
115	Oman	0	2
116	Pakistan	1	0
117	Panama	0	0
118	Paraguay	0	0
119	Peru	0	0
120	Philippines	0	0
121	Poland	0	0
122	Portugal	2	2
123	Qatar	2	2
124	Romania	0	0
125	Russia	0	0
126	Rwanda	1	1
127	Samoa	0	0
128	Saudi Arabia	0	2
129	Senegal	1	1
130	Serbia	0	0
131	Seychelles	0	0
132	Sierra Leone	1	1
133	Singapore	2	2
134	Slovak Republic	2	0
135	Slovenia	2	0
136	Solomon Islands	0	0
137	South Africa	1	0
138	South Korea	2	0
139	Spain	2	2
140	Sri Lanka	0	0
141	St. Vincent and the Grenadines	0	0
142	Sudan	1	0
143	Suriname	0	0
144	Sweden	2	2
145	Switzerland	2	2
146	Tajikistan	0	0
147	Tanzania	1	1
148	Thailand	0	0
149	Timor-Leste	1	0
150	Togo	1	1
151	Tonga	0	0
152	Tunisia	0	0
153	Turkey	0	0
154	Turkmenistan	0	0
155	Uganda	1	1
156	Ukraine	0	0
157	United Arab Emirates	2	2
158	United Kingdom	2	2
159	United States	2	2
160	Uruguay	0	0
161	Uzbekistan	0	0
162	Vanuatu	0	0
163	Venezuela	0	0
164	Vietnam	0	0
165	Yemen	1	0
166	Zambia	1	1

```
------------------------------------------------------------
Os métodos de clusterização tiveram o mesmo resultado em 83.83233532934132% dos caso
s
```

Ambos os métodos classificaram 84% dos países no mesmo cluster, mostrando uma convergência razoavel entre os modelos, enquanto nos demais casos discordaram.

PARTE 4 - Escolha de algoritmos

As etapas do algoritmo de K-means até sua convergência:

- Após a escolha da quantidade de clusters, uma mesma quantidade de pontos são escolhidos aleatoriamente para serem os centroides iniciais da iteração. Essa escolha dos centroides iniciais pode ser realizada de outras formas (K-means++).
- Para cada um dos dados é calculada a distância (que pode ser euclidiana, dentre outras opções) para o centroide, e o dado é atribuido ao cluster com o centroide mais perto
- Após todos os dados terem sido atribuídos, um novo centroide é determinado com base na média da posição de todos os pontos do seu cluster
- Estes passos são repetidos até que o centroide da próxima iteração seja satisfatoriamente igual ao atual, quando ocorre então a convergência

As etapas do algoritmo de K-medoid até sua convergência:

- Similarmente, após a definição da quantidade de clusters, uma mesma quantidade de dados é escolhida para que sejam os medoides iniciais da iteração.
- Novamente, para cada um dos dados é calculada a distância (que pode ser euclidiana, dentre outras opções) para o medoide, e o dado é atribuido ao cluster com o medoide mais perto
- Após todos os dados terem sido atribuídos, um novo ponto é analisado como medoide e as distâncias são novamente medidas. Caso elas sejam menor com ele, ele é o novo medoide deste cluster.
- Este passo é repetido até que o medoide atual seja o que mais adequado para o cluster em questão.

In [26]:
```
# realizando a clusterização pelo método de Kmedoides, com init = 'k-medoids++' par
kmedoide = KMedoids(n_clusters=3, random_state=99, init='k-medoids++')
kmedoide.fit(dataset_tratado)

grupos_kmedoide = kmedoide.predict(dataset_tratado)
resultado_kmedoide = pd.DataFrame(grupos_kmedoide)
```

```python
# nomeando a coluna de agrupamento, e colocando novamente os países na dataframe
resultado_kmedoide = resultado_kmedoide.rename(columns={0: 'Agrupamento'})
resultado_kmedoide['Pais'] = dataset['País']

# printando os resultados
resultado_kmedoide = resultado_kmedoide.sort_values(by='Agrupamento')
pd.set_option('display.max_rows', None)
print(resultado_kmedoide)
```

	Agrupamento	Pais
0	0	Afghanistan
3	0	Angola
31	0	Central African Republic
28	0	Cameroon
25	0	Burkina Faso
26	0	Burundi
21	0	Botswana
17	0	Benin
49	0	Equatorial Guinea
50	0	Eritrea
55	0	Gabon
40	0	Cote d'Ivoire
36	0	Comoros
37	0	Congo, Dem. Rep.
38	0	Congo, Rep.
32	0	Chad
59	0	Ghana
63	0	Guinea
56	0	Gambia
107	0	Myanmar
108	0	Namibia
116	0	Pakistan
88	0	Liberia
94	0	Malawi
80	0	Kenya
81	0	Kiribati
87	0	Lesotho
84	0	Lao
93	0	Madagascar
72	0	Iraq
66	0	Haiti
64	0	Guinea-Bissau
97	0	Mali
99	0	Mauritania
106	0	Mozambique
112	0	Niger
113	0	Nigeria
126	0	Rwanda
166	0	Zambia
165	0	Yemen
147	0	Tanzania
149	0	Timor-Leste
150	0	Togo
155	0	Uganda
137	0	South Africa
142	0	Sudan
132	0	Sierra Leone
129	0	Senegal
60	1	Greece
58	1	Germany
53	1	Finland
29	1	Canada
43	1	Czech Republic
24	1	Bulgaria
54	1	France

51	1	Estonia
33	1	Chile
110	1	Netherlands
85	1	Latvia
91	1	Luxembourg
82	1	Kuwait
42	1	Cyprus
41	1	Croatia
39	1	Costa Rica
123	1	Qatar
122	1	Portugal
121	1	Poland
114	1	Norway
111	1	New Zealand
104	1	Montenegro
96	1	Maldives
98	1	Malta
117	1	Panama
115	1	Oman
77	1	Japan
67	1	Hungary
75	1	Italy
74	1	Israel
73	1	Ireland
68	1	Iceland
95	1	Malaysia
86	1	Lebanon
90	1	Lithuania
131	1	Seychelles
134	1	Slovak Republic
135	1	Slovenia
23	1	Brunei
44	1	Denmark
15	1	Belgium
20	1	Bosnia and Herzegovina
7	1	Australia
4	1	Antigua and Barbuda
13	1	Barbados
11	1	Bahrain
10	1	Bahamas
8	1	Austria
138	1	South Korea
139	1	Spain
133	1	Singapore
145	1	Switzerland
144	1	Sweden
159	1	United States
158	1	United Kingdom
157	1	United Arab Emirates
128	1	Saudi Arabia
130	1	Serbia
160	1	Uruguay
52	2	Fiji
48	2	El Salvador
45	2	Dominican Republic
46	2	Ecuador

47	2	Egypt
62	2	Guatemala
61	2	Grenada
57	2	Georgia
5	2	Argentina
2	2	Algeria
1	2	Albania
6	2	Armenia
9	2	Azerbaijan
14	2	Belarus
12	2	Bangladesh
18	2	Bhutan
19	2	Bolivia
22	2	Brazil
16	2	Belize
27	2	Cambodia
30	2	Cape Verde
127	2	Samoa
125	2	Russia
124	2	Romania
120	2	Philippines
118	2	Paraguay
119	2	Peru
105	2	Morocco
109	2	Nepal
103	2	Mongolia
102	2	Moldova
101	2	Micronesia, Fed. Sts.
100	2	Mauritius
65	2	Guyana
83	2	Kyrgyz Republic
89	2	Libya
92	2	Macedonia, FYR
79	2	Kazakhstan
78	2	Jordan
76	2	Jamaica
71	2	Iran
70	2	Indonesia
69	2	India
35	2	Colombia
34	2	China
143	2	Suriname
151	2	Tonga
146	2	Tajikistan
148	2	Thailand
154	2	Turkmenistan
141	2	St. Vincent and the Grenadines
140	2	Sri Lanka
136	2	Solomon Islands
156	2	Ukraine
152	2	Tunisia
153	2	Turkey
161	2	Uzbekistan
163	2	Venezuela
162	2	Vanuatu
164	2	Vietnam

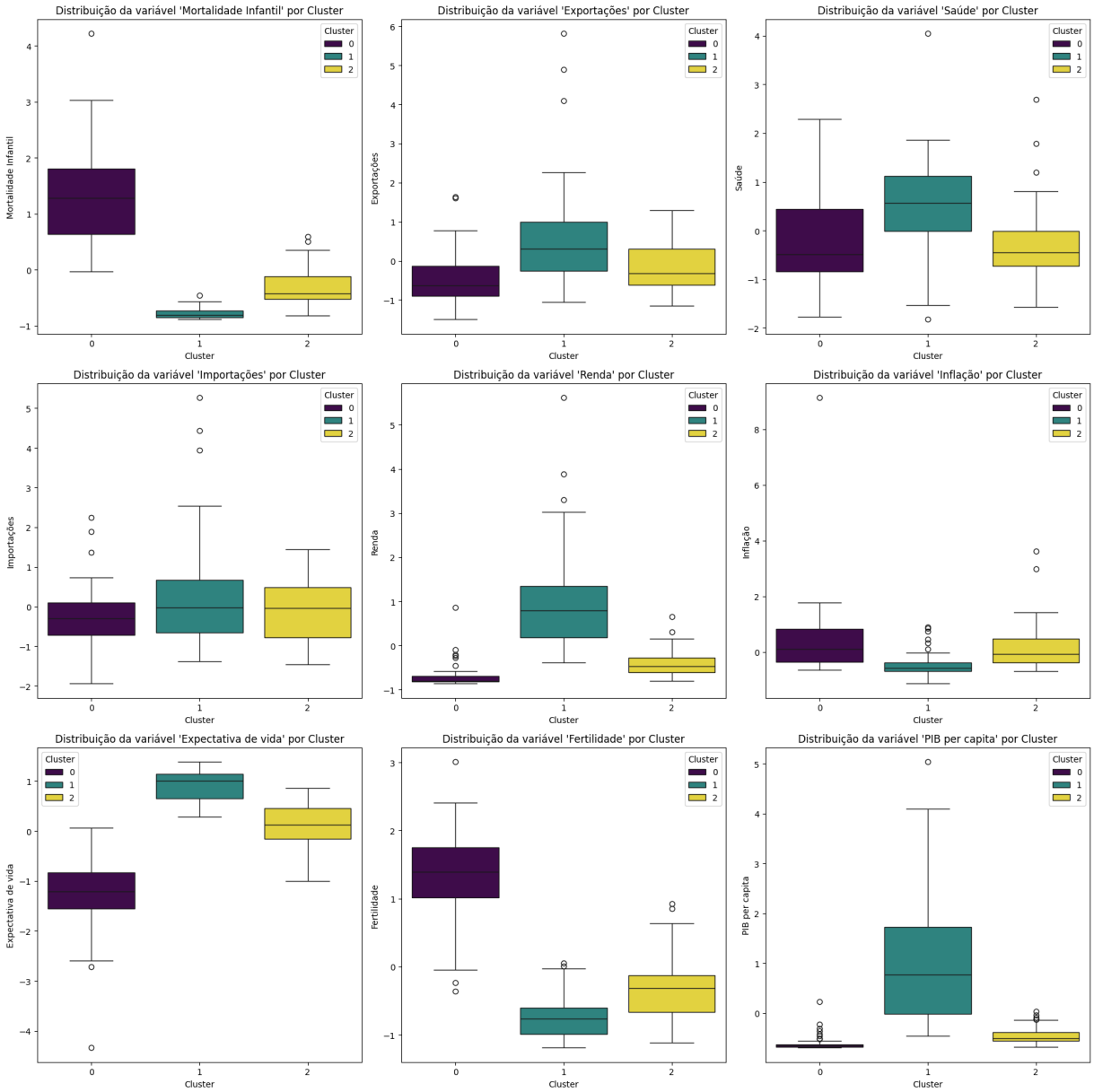
In [27]:
```
# Para o KMedoides, separamos os clusters da seguinte forma para um resultado semân
#    0: "Economias Em vulnerabilidade",
#    1: "Economias Avançada",
#    2: "Economias Emergentes"

nomes_grupos_kmedoide = {
    0: "Economias Em vulnerabilidade",
    1: "Economias Avançada",
    2: "Economias Emergentes"
}

# obtendo o valor de cada variável do medóide
medoides = kmedoide.cluster_centers_
print(f'Os valores de cada variável do medóides são: \n{medoides}')

# obtendo o país que representa o medóide de cada cluster
indices_medoides = kmedoide.medoid_indices_
print(indices_medoides)
paises_medoides = dataset['País'].iloc[indices_medoides]
clusters_medoides = pd.Series(range(len(indices_medoides)), name="Agrupamento")

# Criando um DataFrame com os resultados
df_medoides = pd.DataFrame({
    "Agrupamento": clusters_medoides.map(nomes_grupos_kmedoide),
    "País": paises_medoides.values
})

# Exibindo o resultado
print("-" * 60)
print(f"Os países que representam o medóide de cada cluster são:\n{df_medoides}")
```

```
Os valores de cada variável do medóides são:
[[ 0.83639913 -0.81994613 -0.29419715 -0.73705173 -0.78327176  0.13930796
  -1.26946162  1.64448575 -0.67102583]
 [-0.84983227  0.30336919  0.04174079 -0.02859573  0.68965299 -0.43854531
   1.07644564 -1.13824599  0.49994371]
 [-0.50164289 -0.35891118 -0.73237706  0.11226687 -0.47578376  0.19244389
   0.46741203 -0.51544412 -0.45333648]]
[147 138  76]
------------------------------------------------------------
Os países que representam o medóide de cada cluster são:
                    Agrupamento         País
0  Economias Em vulnerabilidade     Tanzania
1            Economias Avançada  South Korea
2          Economias Emergentes      Jamaica
```

In [ ]:

In [28]:
```
fig, axes = plt.subplots(3, 3, figsize=(18, 18))

# criando boxplots para todas as variáveis numéricas
for i in range(3):
    for j in range(3):
        ax = axes[i, j]
        #selecionando a coluna específica
        coluna = dataset_tratado.columns[i * 3 + j]
```

```
        sns.boxplot(
            data=dataset_tratado.assign(Cluster=resultado_kmedoide['Agrupamento']),
            x='Cluster',
            y=coluna,
            palette="viridis",
            ax=ax,
            hue='Cluster',
        )
        ax.set_title(f"Distribuição da variável '{coluna}' por Cluster")
        ax.set_xlabel("Cluster")
        ax.set_ylabel(coluna)

# ajustando espaçamento para não sobrepor títulos e labels
plt.tight_layout()
plt.show()
```



Sensibilidade de K-means e K-medoids a outliers

A média de um grupo de dados depende do valor específico de cada um deles, enquanto a

mediana de um grupo depende apenas de quantos elementos são maiores e quantos são menores do que o valor central. Logo, a extrapolação de um desses valores externos para um extremo, vai alterar apenas a média mas não a mediana. Por definição, o K-means se baseia nas médias, enquanto o K-medoids se baseia nas medianas. Para uma visualização exagerada, tendo um conjunto de dados [2, 2, 5, 6, 30], a média será 9, enquanto a mediana é 5, note que a média é maior do que todos os números desse conjunto, à excessão do nosso outlier. Se, no lugar do 30 tivessemos um número ainda maior, como 3000, esse fenômeno seria ainda mais exarcebado.

Portanto, o K-means é sensível à outliers por ser definido a partir das médias (centroides) dos pontos.

Robustez do DBScan à presença de outliers

O DBScan (Density-Based Spatial Clustering of Applications with Noise) já indica no próprio nome um dos motivos de ser melhor preparado para tratar de outliers. Por ser um método baseado em densidades de pontos, e não nas distâncias médias discutidas anteriormente, o DBScan identifica as regiões de cada cluster com base nas pontos que estão fora da densidade todos clusters ao mesmo tempo são identificados como ruídos. Portanto, por ignorar pontos que não estão em regiões densas, são, por definição, altamente resistente à outliers.